SVP Solution Report

1 Approach

Basis reduction was a crucial part of optimising the solution to the SVP. A reduced basis significantly shrinks the search space, allowing for a solution in low dimension to be found reasonably fast. As in Külshammer et al (2022) (1), if the basis can no longer be reduced, the shortest basis vector is, therefore, close or equal to the shortest lattice vector.

The approach selected for basis reduction involved employing the LLL algorithm. This algorithm iteratively evaluates a size condition and Lovasz condition until every vector in the basis is satisfied (2). As in Deng, X. (n.d.) (3), for an n-dimensional lattice L, the shortest vector in the LLL-Reduced basis is within $2^{(n-1)/2}$ times $\lambda(L)$ the proximity of the shortest lattice vector. Consequently, this algorithm yields, either a reasonably accurate approximation or, for lower dimensions, an exact value for the shortest vector.

1.1 Running Time

An advantageous characteristic of the LLL algorithm is its ability to yield an approximate solution in reasonable polynomial time and space. Specifically, it has been shown that the algorithm executes in $\mathcal{O}(n^5 \times m \times (\log Y)^3)$ time on a lattice $\mathbb{Z}^m$ with a basis: $b_1, \ldots, b_n$ where $Y \in \mathbb{Z}_{\geq 2}$. This is a vast improvement on the exponential running time associated with simple enumeration methods and is therefore a significantly more viable option when solving the SVP for non-trivial dimensions.

The space complexity of LLL is $\mathcal{O}(n^2)$. This modest polynomial complexity is attributed to the necessity of storing an $n \times n$ basis of vectors to perform matrix operations and facilitates efficient memory use as demonstrated below.

2 Error Handling and Bounds Checking

To ensure robust error-handling of the input, a range of techniques were employed. The system enforces the following conditions:

- Consistency in the number of dimensions across vectors.
- Equality between the number of vectors and the specified dimension.
- Each opening bracket necessitates a closing one.
- Adequate spacing between input values.

If any of these conditions are not met, the program responds with an appropriate error message and gracefully exits, ensuring only valid basis matrices are processed.

3 Memory Allocation

Due to the complexity of the matrix operations required in the LLL algorithm, careful attention was given to mitigating memory wastage in the implementation. To optimise memory usage, memory was allocated dynamically to the structures representing the matrices and the vectors within them. This allowed for flexibility on input sizes, meaning the program was adaptable to different dimensions without wasting memory.

Upon completion of each process, memory allocated to structures no longer being used was freed to prevent memory leakage, thereby reducing overall memory consumption throughout execution.

4 Measuring Correctness and Efficiency

4.1 Time Consumption

The execution time of the program was tested up to 6 dimensions with a set of 3 basis matrices using the inbuilt gettimeofday() function. Execution time was reasonably consistent amongst dimensions, with only marginal increases in execution speed with each increase of dimension, as visualised in Figure 1. A great deal of this execution time was likely due to background processes, so in reality, the execution speed is likely lower. This reaffirms the fact that the LLL reduction produces a result in acceptable polynomial time.

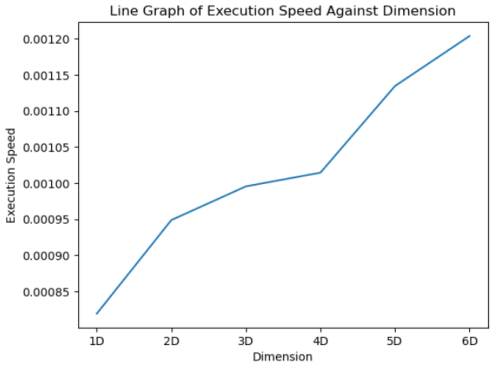

Fig. 1: Execution speeds against dimension

4.2 Accuracy

The application underwent rigorous testing across a range of low-dimensional lattices. Figure 2 shows the lattices used in testing, comparing the expected results with those produced by the program. The conducted tests yielded a 100% accuracy rate for the program, affirming design success.

Basis	Known Solution	Calculated Result
[3 1] [4 5]	3.162278	3.162278
[8 1] [8 2]	1	1
[1 0 0] [0 1 0] [0 0 1]	1	1
[1 1 1] [-1 0 2] [3 5 6]	1	1
[1 0 0] [4 2 15] [0 0 3]	1	1
[5 6 2 1] [7 3 1 4] [7 8 4 2] [3 2 0 6]	3.162278	3.162278
[4 2 7 9] [0 0 1 2] [9 9 9 1] [1 2 1 2]	2.236068	2.236068
[1 1 1 1 1] [2 2 2 2 2] [3 3 3 3 3] [4 4 4 4 4] [5 5 5 5 5]	2	2
[1 5 6 2 1] [7 2 4 1 3] [8 3 2 3 4] [1 0 0 2 1] [6 2 2 5 2]	2.236068	2.236068

Fig. 2: Test lattices

4.3 Memory Usage

The Valgrind toolkit was utilised for memory usage analysis during program execution. To measure heap utilisation, six dimensions of bases were applied to the same set of three testing bases employed during execution-time analysis. Heap utilisation was assessed by measuring counts of memory allocations and deallocations. The examination across all test bases revealed complete heap utilisation, with every allocation being followed by a subsequent deallocation, indicating the absence of memory leaks. Figure 3 provides an illustrative example.

Additionally, the mean total bytes allocated per dimension was examined. The results indicated an exponential trend, visualised in Figure 4. It's important to note that the program is only used for solving low-dimensional instances of the SVP. Consequently, memory usage remains well within acceptable limits. To illustrate, the algorithm allocated 45,000 bytes to process a six-dimensional basis, which is quite satisfactory for the problem.

```
joshua_bourton@3:/mnt/c/Users/Owner/Documents/Comp Sci/Systems programming/Lattices
Summative (50%)/SVP$ valgrind --leak-check=full ./runme [1 5 6 2 1] [7 2 4 1 3] [8 3
 2 3 4] [1 0 0 2 1] [6 2 2 5 2]
==644== Memcheck, a memory error detector
==644== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==644== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==644== Command: ./runme [1 5 6 2 1] [7 2 4 1 3] [8 3 2 3 4] [1 0 0 2 1] [6 2 2 5 2]
==644==
==644== error calling PR_SET_PTRACER, vgdb might block
Elapsed time: 0.074050 seconds
==644==
==644== HEAP SUMMARY:
==644==     in use at exit: 0 bytes in 0 blocks
==644==   total heap usage: 461 allocs, 461 frees, 27,044 bytes allocated
==644==
==644== All heap blocks were freed -- no leaks are possible
==644==
==644== For lists of detected and suppressed errors, rerun with: -s
==644== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
joshua_bourton@3:/mnt/c/Users/Owner/Documents/Comp Sci/Systems programming/Lattices
Summative (50%)/SVP$
```

Fig. 3: Valgrind tool usage on a test basis

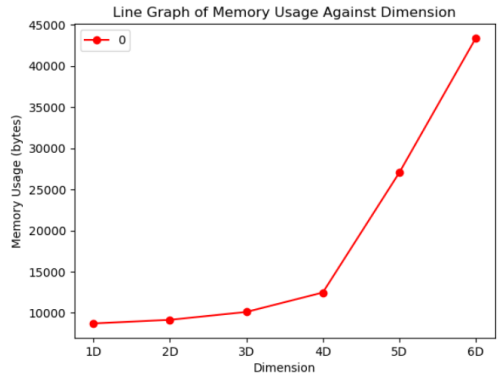

Fig. 4: Memory allocation across dimensions

REFERENCES

[1] Külshammer, J., & Polách, J. (2022). Lattice Basis Reduction Using LLL Algorithm with Application to Algorithmic Lattice Problems. Retrieved from https://uu.diva-portal.org/smash/get/diva2:1641300/FULLTEXT01.pdf

[2] Ortega, J. (2022). CSUSB ScholarWorks CSUSB ScholarWorks Electronic Theses, Projects, and Dissertations Office of Graduate Studies LATTICE REDUCTION ALGORITHMS LATTICE REDUCTION ALGORITHMS. https://scholarworks.lib.csusb.edu/cgi/viewcontent.cgi?article=2675&context=etd

[3] Deng, X. (n.d.). An Introduction to Lenstra-Lenstra-Lovász Lattice Basis Reduction Algorithm. Retrieved from https://math.mit.edu/~apost/courses/18.204-2016/18.204_Xinyue_Deng_final_paper.pdf

[4] El, F.-E. (2019). Complexity of the LLL algorithm Seminar on lattices. https://sp2.uni.lu/wp-content/uploads/sites/66/2019/07/Complexity-of-LLL-Fatima-El-Orche.pdf